

Completed Task

Status: Zakonczone - multi-currency wdrozone i zweryfikowane

Task: Multi-currency na podstawie lokalizacji/locale

Last Updated: 2026-06-13

Completed: 2026-06-13

Cel

Wprowadzić centralną obsługę walut w aplikacji:

- Polska / `pl` -> `PLN`
- Anglia / UK / `en` -> `GBP`
- Portugalia / `pt` -> `EUR`
- fallback -> `GBP` lub skonfigurowana waluta domyślna

Waluta ma być rozwiązywana centralnie, udostępniana frontendowi przez Inertia i używana przy tworzeniu nowych leadów, klientów, zamówień domen, checkoutów, planów SaaS oraz publicznych cen. Istniejące dokumenty finansowe zachowują własną walutę historyczną.

Delta Analysis

Anchor files

- `app/Http/Middleware/HandleInertiaRequests.php` — obecne rozwiązywanie `locale` z sesji / `Accept-Language`; najlepszy punkt do udostępnienia `currency`.
- `config/languages.php` — lista obsługiwanych języków `pl`, `en`, `pt`.
- `routes/web/public.php` — obecny przełącznik `/lang/{locale}`.
- `app/Filament/Pages/PaymentSettingsPage.php` — istniejąca konfiguracja `payment_currency`.
- `app/Models/Setting.php` — storage ustawień globalnych.
- `app/Actions/CreateLeadAction.php` — obecnie hardcoded `country = GB`, `currency = GBP`.
- `app/Actions/Domain/CreateDomainOrderAction.php` — obecnie hardcoded `currency = GBP`.
- `app/Actions/Domain/CheckDomainAvailabilityAction.php` — fallback `currency = GBP`.
- `app/Services/Domain/DomainPricingService.php` — źródło cen domen z `domain_price_list`.
- `database/migrations/2026_05_29_000001_create_domain_price_list_table.php` — lista cen domen ma pojedynczą walutę per TLD.
- `app/Http/Controllers/Portal/BillingController.php` — plany SaaS obecnie zwracane jako `GBP`.
- `app/Services/Billing/PlanService.php` i `app/Models/Plan.php` — ceny planów w pensach, bez waluty.
- `resources/js/Pages/Portal/Billing/Index.jsx` — cena planu hardcoded jako `£`.
- `resources/js/Pages/Domains/*.jsx` i `resources/js/Pages/Portal/Domains/*.jsx` — ręczne mapowania symboli walut.

- `resources/js/Pages/Portal/*` — część stron używa `Intl.NumberFormat`, część dokleja symbol ręcznie.
- `app/Filament/Resources/*Resource.php`, widżety i Blade reports — wiele `money('GBP')`, `prefix('£')`, `number_format`.

Luki

- Brak centralnego `CurrencyResolver`.
- Brak `config/currencies.php`.
- Brak wspólnego helpera do formatowania walut w PHP i JS.
- Brak ceny per waluta dla planów SaaS i publicznych pakietów usług.
- Domeny mają tylko jedną cenę per TLD, więc nie da się poprawnie pokazać `PLN/EUR/GBP` bez migracji albo przelicznika.
- Stripe subskrypcje wymagają osobnych Price ID per waluta.
- PayU powinno być ograniczone/zweryfikowane pod obsługiwane waluty, szczególnie `PLN`.

Ryzyka

- Nie wolno zmieniać waluty istniejących faktur, płatności, ofert, kontraktów ani zamówień po utworzeniu.
- Automatyczne przeliczanie kwot historycznych zepsuje raporty i księgowość.
- Stripe Price ID jest powiązane z walutą; samo ustawienie `currency` w Checkout nie działa przy istniejącym `price`.
- Raporty agregujące wiele walut nie mogą sumować kwot bez grupowania po walucie albo waluty bazowej.
- Ceny publiczne zapisane jako tekst z `£` wymagają osobnej normalizacji.

Rekomendowana architektura

1. Centralna konfiguracja walut

Utworzyć `config/currencies.php`:

- `default` -> `GBP`
- `supported` -> `GBP, EUR, PLN`
- `locale_map` -> `en: GBP, pl: PLN, pt: EUR`
- `country_map` -> `GB: GBP, PL: PLN, PT: EUR`
- `metadata`: `symbol`, `decimal digits`, `display locale`, `minor unit factor`.

2. CurrencyResolver

Utworzyć `app/Services/Currency/CurrencyResolver.php`.

Kolejność rozwiązywania:

1. waluta z aktualnego `locale`;
2. `Setting::get('payment_currency')`;
3. fallback z `config/currencies.default`.

Na start bez GeolP i bez widocznego przełącznika waluty. Waluta ma być konsekwencją języka/locale.

3. Inertia shared props

Rozszerzyć `HandleInertiaRequests` o:

- `currency`
- `available_currencies`
- `currency_settings`, np. symbol i locale do formatowania

Frontend dostaje jedną prawdę zamiast lokalnych mapowań `GBP ? £ : EUR ? €`.

4. Helpery formatowania

Backend:

- `app/Support/Money.php` lub `app/Services/Currency/MoneyFormatter.php`
- formatowanie przez `NumberFormatter/Intl`, fallback do symbolu z configu.

Frontend:

- `resources/js/Utils/currency.js` albo hook `useCurrency()`
- `formatCurrency(amount, currency, locale)`
- wymiana ręcznych `£, currency === 'EUR' ? ...` na helper.

5. Dane cenowe: price book, nie kursy walut

Preferowana strategia: przechowywać ceny per waluta, ale generować je z ceny bazowej GBP według reguły biznesowej.

Reguła przeliczenia:

- źródło: cena bazowa w GBP;
- kurs dla waluty docelowej zaokrąglany w górę do najbliższej dziesiątej;
- przykłady: `4.93 -> 5.00`, `5.11 -> 5.20`;
- kwota końcowa zapisywana w price book, aby checkout/oferta/faktura miały stabilną cenę.

Nie przeliczać dynamicznie istniejących ofert/faktur. Dokument finansowy ma zapisaną własną walutę i kwotę.

6. SaaS plans

Migracja `plans`:

- dodać `currency` tylko jeśli plan ma być single-currency, albo lepiej:
- stworzyć `plan_prices`:
 - `plan_id`
 - `currency`
 - `interval` (`monthly`, `yearly`)
 - `amount_minor`
 - `stripe_price_id`
 - `unique: plan_id + currency + interval`

Zaktualizować:

- `PlanService` -> zwraca cenę dla bieżącej waluty.
- `BillingController::index()` -> `currency` z resolvera.
- `BillingController::checkout()` -> wybiera `stripe_price_id` dla waluty.
- `PlanResource` -> zarządzanie cenami per waluta.
- `PlanSeeder` -> ceny GBP/EUR/PLN.

7. Domeny

Dwie opcje:

- Etap 1 szybki: rozszerzyć `domain_price_list` do wielu rekordów per `tld + currency`; usunąć unique tylko na `tld`, dodać unique `tld + currency`.
- Etap 2 lepszy: stworzyć `domain_price_overrides` lub przebudować price list na price book z wholesale base + retail per currency.

Rekomendacja: etap 1, bo pasuje do obecnej architektury i ogranicza blast radius.

Zmiany:

- `DomainPricingService::getPriceForTld($tld, $currency)`
- `getAllActivePrices($currency)`
- `calculateRetailPrice($tld, $years, $action, $currency)`
- `CreateDomainOrderAction` zapisuje walutę z resolvera.
- `CheckDomainAvailabilityAction` zwraca ceny w aktualnej walucie.
- UI domen korzysta z `formatCurrency`.

8. Leads, clients, projects, quotes, invoices, contracts

Nowe rekordy:

- domyślna waluta z `CurrencyResolver`;
- zachować możliwość ręcznej zmiany w adminie/Filament.

Istniejące rekordy:

- bez migracji wartości;
- waluta zostaje taka, jaka jest w kolumnie `currency`.

Filament:

- selecty walut biorą opcje z `config/currencies.php`;
- `prefix('£')` i `money('GBP')` zastąpić helperami/closure zależnymi od rekordu.

9. Publiczne ceny usług i kalkulatory

Obecnie część cen jest tekstem typu `£799, £149/mo`.

Plan bezpieczny:

- dla CMS/ServiceItem wprowadzić pola strukturalne: `price_from_amount`, `price_from_currency`, opcjonalnie `price_from_period`;

- albo dla treści marketingowych dodać JSON per waluta, np. `price_from: { GBP: "...", EUR: "...", PLN: "..." }`.

Rekomendacja:

- tam gdzie cena wpływa na decyzję zakupową: strukturalna kwota + waluta;
- tam gdzie to copy marketingowe: locale/currency keyed string.

10. Płatności

Stripe:

- invoice/domain checkout może nadal używać dynamicznego `price_data.currency`;
- SaaS subscriptions muszą mieć `stripe_price_id` per waluta.

PayU:

- obsługiwać `GBP`, `EUR`, `PLN`, jeśli POS/umowa PayU ma włączoną konfigurację multi-currency;
- uwzględnić, że dostępność metod płatności zależy od waluty: karty szeroko, przelewy/bank transfery głównie `CZK`, `EUR`, `PLN`;
- fallback płatności dla `GBP/EUR/PLN`: Stripe.

11. Raporty i dashboardy

- Raporty finansowe grupować po `currency`.
- Nie sumować `GBP + EUR + PLN` w jednej liczbie.
- Dashboardy typu MRR/Revenue: albo pokazać sekcje per waluta, albo dodać walutę bazową i kursy raportowe jako osobny etap.

Plan wdrożenia

Faza 1 — Fundament

- ☒ Dodać `config/currencies.php`.
- ☒ Dodać `CurrencyResolver`.
- ☒ Dodać `MoneyFormatter`/helper PHP.
- ☒ Dodać frontendowy `formatCurrency` / `useCurrency`.
- ☒ Udostępnić `currency` i `available_currencies` w Inertia.
- ☒ Nie dodawać publicznego przełącznika waluty w MVP; waluta wynika z `locale`.
- ☒ Opcjonalnie zostawić wewnętrzną możliwość override tylko w adminie/dokumentach finansowych.

Faza 2 — Nowe rekordy i UI podstawowe

- ☒ Podpiąć resolver do `CreateLeadAction`.
- ☒ Podpiąć resolver do formularzy/akcji tworzących klientów, projekty, oferty, faktury i kontrakty.
- ☒ Ujednolicić opcje walut w Filament przez config.
- ☒ Wymienić najbardziej widoczne hardcoded `£` w portalu klienta i publicznych stronach.

Faza 3 — Domeny

- ☒ Zmienić model cen domen na `tld + currency`.
- ☒ Zaktualizować `DomainPricingService`.
- ☒ Zaktualizować `CheckDomainAvailabilityAction` i `CreateDomainOrderAction`.
- ☒ Zaktualizować publiczne i portalowe strony domen.
- ☒ Dodać testy dla `pl -> PLN`, `en -> GBP`, `pt -> EUR`.

Faza 4 — SaaS billing

- ☒ Dodać `plan_prices`.
- ☒ Dodać generator/synchronizację cen walutowych z GBP według reguły zaokrąglania kursu w górę do 0.10.
- ☒ Zaktualizować `PlanService`.
- ☒ Zaktualizować `BillingController` i Stripe Checkout.
- ☒ Zaktualizować `PlanResource`.
- ☒ Zaktualizować seeder planów i env/config dla Stripe Price IDs per waluta.

Faza 5 — Raporty, PDF, email, dashboard

- ☒ Wymienić `money('GBP')`, `prefix('£')`, ręczne `number_format` w finansach.
- ☒ Raporty agregować per waluta.
- ☒ PDF faktury i e-maile formatować przez helper.
- ☒ Dashboardy pokazywać walutę rekordu lub grupowanie per currency.

Faza 6 — Walidacja

- ☒ PHPUnit: `CurrencyResolverTest`.
- ☒ PHPUnit: domeny i billing checkout dla `GBP/EUR/PLN`.
- ☒ Feature tests dla Inertia props.
- ☒ Testy regresji dla istniejących faktur/ofert — waluta historyczna bez zmian.
- ☒ `php artisan pint`
- ☒ `npm run lint`
- ☒ `php artisan test`
- ☒ `./.github/scripts/check-translations.sh`

Decyzje

1. Waluta wynika z aktualnego `locale`: `pl -> PLN`, `en -> GBP`, `pt -> EUR`.
2. Rekomendacja MVP: bez widocznego przełącznika waluty; przełącznik języka zmienia walutę.
3. Ceny docelowe generowane z GBP po kursie zaokrąglonym w górę do najbliższej dziesiątej, a potem zapisywane w price book.
4. PayU próbujemy obsłużyć dla `GBP`, `EUR`, `PLN`, z fallbackiem do Stripe i walidacją metod dostępnych dla waluty.

Faza 1 - wykonane

Backend

- `config/currencies.php` - centralna mapa walut: `en -> GBP`, `pl -> PLN`, `pt -> EUR`, metadata symboli i formatowania.
- `app/Services/Currency/CurrencyResolver.php` - resolver waluty z `locale`, z fallbackiem do `payment_currency` i `APP_CURRENCY/GBP`.
- `app/Services/Currency/MoneyFormatter.php` - formatowanie kwot po stronie PHP.
- `app/Services/Currency/CurrencyPriceCalculator.php` - fundament pod przeliczanie z GBP: kurs zaokrąglany w górę do najbliższego `0.10`.
- `app/Http/Middleware/HandleInertiaRequests.php` - shared props: `currency`, `available_currencies`, `currency_settings`.

Frontend

- `resources/js/utils/currency.js` - `normalizeCurrency`, `getCurrencyMeta`, `formatCurrency`.
- `resources/js/Hooks/useCurrency.js` - hook oparty o Inertia shared props.

Dodatkowa poprawka walidacyjna

- `resources/views/app.blade.php` - loader Vite wybiera `.tsx`, gdy komponent Inertia istnieje jako TSX. Naprawia `Services/ServicePage`, który miał plik `ServicePage.tsx`, a root Blade wymuszał `.jsx`.

Testy i walidacja

- `php artisan test --filter=Currency` - OK, 8 testów / 18 asercji.
- `npm test -- currency.test.js` - OK, 3 testy.
- `vendor\bin\pint --dirty` - OK, poprawił formatowanie zmienionych plików PHP.
- `npm run build` - OK.
- `php artisan test --filter=ServicePageControllerTest` - OK, 13 testów / 127 asercji po poprawce loadera `.tsx`.
- `npm run lint` - było zablokowane przez konfigurację projektu: ESLint 10 oczekiwał `eslint.config.js`, a repo miało `.eslintrc.json`; naprawione po Fazie 2.
- `bash ./github/scripts/check-translations.sh` - zablokowane lokalnie: `bash` nie jest dostępny w tej sesji PowerShell.
- `php artisan test` - częściowo zablokowane przez istniejące 404 dla ekranów `auth/profile (/login, /register, /forgot-password, /verify-email, /confirm-password, /profile)`; testy walutowe i `ServicePage` przechodzą.

Faza 2 - wykonane

Backend i modele

- `app/Models/Concerns/DefaultsCurrency.php` - trait ustawiający domyślną walutę z `CurrencyResolver` przy tworzeniu nowych rekordów.
- Modele `Lead`, `Client`, `Project`, `Quote`, `Invoice`, `Contract`, `Payment` używają traitu `DefaultsCurrency`.
- `app/Actions/CreateLeadAction.php` - nowe leady i nowi klienci dostają walutę z `locale`, a jawnie podana waluta wygrywa z fallbackiem.

- `CurrencyResolver::countryForLocale()` + `locale_country_map` - klient tworzony z leada dostaje domyslany kraj z locale (`pl` -> `PL`, `pt` -> `PT`, `en` -> `GB`).
- `CreateDomainOrderAction` zapisuje walute z aktualnej ceny TLD zamiast stalego `GBP`; pelny price book domen zostaje w Fazie 3.

Filament

- Dodano `app/Filament/Support/Currency.php` jako wspolny helper dla opcji, symboli, formatowania i kolumn `money()`.
- Glowne zasoby CRM/finansowe (`Lead`, `Client`, `Project`, `Quote`, `Invoice`, `Contract`, `Payment`) pobieraja opcje walut z `config/currencies.php`.
- Widoki szczegolow Blade dla leada, klienta, oferty, faktury i kontraktu formatuja kwoty przez `MoneyFormatter`.
- `PaymentSettingsPage` korzysta z centralnych opcji walut.

Frontend

- Portal klienta (`Invoices`, `Invoice`, `PayInvoice`, `PaymentResult`, `Projects`, `Project`, `Quotes`, `Quote`, `Contracts`, `Contract`, `Dashboard`, `Billing`) uzywa `useCurrency()`.
- Publiczne i portalowe strony domen przestaly zgadywac symbole walut w UI; formatter uzywa waluty rekordu/zamowienia/ceny TLD.
- Publiczna strona domen nie pokazuje juz statycznych cen pakietow w GBP tam, gdzie nie ma jeszcze strukturalnego price booka.
- Lead details w Inertia pokazuje wartosc przez `useCurrency()`.

Testy i walidacja

- Dodano `tests/Feature/Currency/CurrencyDefaultsTest.php`.
- `vendor/bin/pint --dirty` - OK, poprawil formatowanie zmienionych plikow PHP.
- `php artisan test --filter=Currency` - OK, 11 testow / 25 asercji.
- `php artisan test tests/Feature/Currency/CurrencyDefaultsTest.php` - OK, 3 testy / 7 asercji.
- `php artisan test tests/Feature/CalculatorLeadTest.php` - OK, 6 testow / 19 asercji.
- `npm test -- currency.test.js` - OK, 3 testy.
- `npm run build` - OK.
- `npm run lint` - OK po dodaniu `eslint.config.js` i dopasowaniu ESLint do wspieranego przez pluginy React major 9.
- `bash ./github/scripts/check-translations.sh` - nadal zablokowane lokalnie: `bash` nie jest dostepny w tej sesji PowerShell.

Pozostawione na kolejne fazy

- Kalkulatory i strukturalne publiczne ceny uslug - osobny price book z punktu architektury 9.
- SaaS plany i Stripe Price ID per waluta - Faza 4.
- Widzety, raporty, PDF-y i agregacje finansowe per waluta - Faza 5.

Faza 3 - wykonane

Baza danych i price book domen

- Dodano migracje:
 - `2026_06_12_000001_allow_domain_price_list_multi_currency.php` - usuwa unikalność samego `tld` i dodaje `unique tld + currency`.
 - `2026_06_12_000002_add_currency_to_domain_renewals_table.php` - dodaje `currency` do renewal records.
- `DomainPriceList` normalizuje `tld` do lowercase i `currency` do uppercase.
- `DomainPriceListSeeder` tworzy ceny dla `GBP`, `EUR`, `PLN`; `GBP` jest baza, a `EUR/PLN` są wyliczane przez `CurrencyPriceCalculator` z kursem zaokrąglanym w górę do `0.10`.

Pricing i przepływy domen

- `DomainPricingService` obsługuje `getPriceForTld($tld, $currency)`, `calculateRetailPrice(..., $currency)` i `getAllActivePrices($currency)`.
- Pricing wybiera walutę z `CurrencyResolver`, a jeśli cena dla locale nie istnieje, bezpiecznie wraca do domyślnej waluty `GBP`.
- `getAllActivePrices()` jest teraz DB-agnostic i nie używa już MySQL-only `FIELD()`, więc testy działają na SQLite.
- `CheckDomainAvailabilityAction`, `CreateDomainOrderAction`, publiczny order form i portalowy order form używają ceny w aktualnej walucie.
- Renewal records tworzone przez `RenewDomainAction`, `DomainRenewalService` i `RegisterDomainJob` zapisują walutę zgodną z ceną/orderem.
- `ManualDomainRegistrarService` pobiera ceny przez `DomainPricingService`, zamiast wybierać pierwszy aktywny rekord TLD.
- Sync Openprovider ograniczono do bazowej waluty price listy, żeby hurtowe ceny z API nie nadpisywały wierszy `EUR/PLN` jak wartości `GBP`.

Filament

- `DomainPriceListResource` ma waluty z centralnego configu i walidację unikalności pary `tld + currency`.
- Prefixy kwot w formularzu listy cen domen reagują na wybraną walutę.
- `DomainPriceListResource` i `DomainOrderResource` formatują kwoty przez walutę rekordu zamiast stałego `GBP`.

Testy i walidacja

- Rozszerzono `tests/Unit/Domain/DomainPricingServiceTest.php` o wybór `PLN`, `EUR`, fallback do `GBP` i listę cen per waluta.
- Rozszerzono `tests/Feature/Domain/DomainPurchaseFlowTest.php` o zamówienie domeny w `PLN` przy locale `pl`.
- `vendor\bin\pint --dirty` - OK.
- `php artisan test --filter=DomainPricingServiceTest` - OK, 22 testy / 43 asercje.
- `php artisan test --filter=DomainPurchaseFlowTest` - OK, 19 testów / 34 asercje.
- `php artisan test --filter=Domain` - OK, 77 testów / 159 asercji, 10 sandbox testów OpenProvider pominiętych z powodu braku credentials.
- `php artisan test --filter=Currency` - OK, 17 testów / 40 asercji.
- `npm test -- currency.test.js` - OK, 3 testy.
- `npm run lint` - OK.

- `npm run build` - OK; Vite pokazuje tylko istniejące ostrzeżenia deprecation dla pluginu React/rolldown.
- `git diff --check` - OK; tylko ostrzeżenia Git o normalizacji CRLF w istniejących plikach.

Pozostawione na kolejne fazy

- Strukturalny price book dla kalkulatorów/publicznych cen usług - dalsza faza po SaaS billing.
- Widżety, raporty, PDF-y i agregacje finansowe per waluta - Faza 5.

Faza 4 - wykonane

Baza danych i model

- Dodano `database/migrations/2026_06_13_000001_create_plan_prices_table.php`.
- Nowa tabela `plan_prices` ma:
 - `plan_id`
 - `currency`
 - `interval` (`monthly`, `yearly`)
 - `amount_minor`
 - `stripe_price_id`
 - `is_active`
 - `unique plan_id + currency + interval`.
- Migracja backfilluje istniejące ceny z `plans.price_monthly`, `plans.price_yearly` i legacy Stripe IDs jako rekordy `GBP`.
- Dodano model `App\Models\PlanPrice` i relacje `Plan::planPrices()`.

PlanService i cennik

- `PlanService::getPlans($currency)` zwraca plany z cennikiem dla waluty wynikającej z locale.
- Fallback ceny: waluta requestu -> domyślna waluta `GBP` -> pierwsza aktywna cena planu.
- Cache planów jest teraz per waluta, np. `saas_plans_pln`, i jest czyszczony przez `PlanService::clearCache()`.
- Dodano `PlanService::getCheckoutPrice()`, `stripePriceIdFor()` i `findPlanSlugByStripePriceId()` dla checkoutu i webhooków.

Stripe checkout i webhook

- `BillingController::index()` pokazuje ceny planów w walucie requestu, a nie stale `GBP`.
- `BillingController::checkout()` wybiera `PlanPrice` po planie, walucie i interwale, wymaga aktywnej płatnej ceny oraz Stripe Price ID.
- Checkout metadata zawiera `plan_price_id`, `currency` i `interval`.
- `SubscriptionWebhookController` mapuje Price ID przez `plan_prices` albo nowy config Stripe, a nie tylko legacy `STRIPE_PRICE_PRO_MONTHLY` / `STRIPE_PRICE_AGENCY_MONTHLY`.
- Webhook zapisuje też `stripe_subscription_id` i `stripe_subscription_status`.

Config, seeder, admin

- `config/services.php` ma nowe klucze `services.stripe.prices.{plan}.{currency}.{interval}` dla `basic`, `pro`, `agency` oraz `GBP/EUR/PLN`.
- Zachowano legacy fallback:
 - `STRIPE_PRICE_PRO_MONTHLY`
 - `STRIPE_PRICE_AGENCY_MONTHLY`
- `PlanSeeder` tworzy `plan_prices` dla `GBP`, `EUR`, `PLN`, bazując na cenach `GBP` i `CurrencyPriceCalculator`.
- `PlanResource` ma price book jako repeater, z osobnymi cenami i Stripe Price ID per waluta/interwal.
- Portal billing wysyla do checkoutu jawny `interval: monthly` i ma style dla planu `basic`.

Testy i walidacja

- Dodano `tests/Unit/Billing/PlanServiceTest.php`.
- Rozszerzono `tests/Feature/Portal/PortalBillingAccessTest.php`.
- `vendor\bin\pint --dirty` - OK.
- `php artisan test --filter=Billing` - OK, 9 testow / 45 asercji.
- `php artisan test --filter=Currency` - OK, 23 testy / 70 asercji.
- `npm run lint` - OK.
- `npm run build` - OK; Vite pokazuje tylko istniejace ostrzezenia deprecation/plugin timing.

Pozostawione na kolejne fazy

- Strukturalny price book dla kalkulatorow i publicznych cen uslug.
- Widzety, raporty, PDF-y i agregacje finansowe per waluta.
- Pelne testy realnego Stripe Checkout nadal wymagaja skonfigurowanych Stripe Price ID w env/panelu Stripe.

Faza 5 - wykonane

Wspolne formatowanie i agregacje

- Dodano `App\Services\Currency\CurrencySummaryFormatter` do sumowania rekordow per `currency` i formatowania zbiorczych kwot jako osobnych walut.
- Raport faktur agreguje statusy per waluta zamiast sumowac `GBP + EUR + PLN` do jednej liczby.
- Raporty leadow/projektow i eksport leadow pokazuja walute rekordu.

PDF, e-maile i komunikaty

- PDF faktury uzywa `MoneyFormatter` dla pozycji, subtotal, VAT, total, paid i amount due.
- E-maile faktur, ofert, zaakceptowanych ofert, kontraktow, potwierdzen platnosci i nowych leadow formatuja kwoty przez `MoneyFormatter`.
- SMS-y potwierdzenia platnosci Stripe/PayU uzywaja formattera waluty platnosci.
- Preview szablonow e-mail dobiera przykladowa walute z locale (`en -> GBP`, `pl -> PLN`, `pt -> EUR`).
- Placeholdery kontraktow `{{project.budget}}` i `{{contract.value}}` zwracaja sformatowana kwote z waluta rekordu.

Filament i dashboard

- WIDGETY tabelowe projektów, faktur i leadów używają waluty rekordu zamiast GBP.
- `StatsOverviewWidget`, `DomainOrderStatsWidget`, `RevenueChartWidget`, `PipelinePage`, `ConversionReportPage` i sekcje finansowe projektu grupują wartości per currency.
- `SaasMetricsWidget` liczy MRR/ARR z `plan_prices`, z obsługą waluty i interwału subskrypcji.
- Dodano pola subskrypcji do `businesses`: `plan_price_id`, `stripe_subscription_currency`, `stripe_subscription_interval`.
- `SubscriptionWebhookController` zapisuje price/currency/interval subskrypcji z checkout metadata albo Stripe Price ID.
- Admin kalkulatora cen używa centralnych opcji walut, dynamicznych prefiksów i waluty rekordu w tabelach.

Testy i walidacja

- Dodano `tests/Unit/Currency/CurrencySummaryFormatterTest.php`.
- Rozszerzono `tests/Feature/ReportInvoicesTest.php` o regresje agregacji faktur per waluta.
- Rozszerzono `tests/Unit/Billing/PlanServiceTest.php` o odnajdywanie `PlanPrice` po Stripe Price ID z configu.
- `vendor/bin/pint --dirty` - OK.
- `php artisan test --filter=Currency` - OK, 26 testów / 78 asercji.
- `php artisan test --filter=Billing` - OK, 9 testów / 46 asercji.
- `php artisan test --filter=ReportInvoicesTest` - OK, 8 testów / 18 asercji.
- `php artisan test --filter=Domain` - OK, 77 testów / 159 asercji, 10 sandbox testów OpenProvider pominiętych z powodu braku credentials.
- `npm run lint` - OK.
- `npm run build` - OK; Vite pokazuje tylko istniejące ostrzeżenia deprecation dla pluginu React/rolldown.
- `git diff --check` - OK; tylko ostrzeżenia Git o normalizacji CRLF w istniejących plikach.

Pozostawione na kolejne fazy

- Strukturalny price book dla publicznych cen usług został zrealizowany w Fazie 7.
- Publiczne seedery i frontendowe fallbacki marketingowe zostały oczyszczone z hardcoded £ w Fazie 8.
- OpenProvider diff nadal pokazuje hurtowe ceny w bazowej walucie importu.

Faza 6 - wykonane

Pelna walidacja

- `php artisan test` - 411 testów passed, 10 skipped, 6 failed.
- Failujące testy są znane i dotyczą brakujących ekranów routingu auth/profile:
 - `/login`
 - `/verify-email`
 - `/confirm-password`
 - `/forgot-password`
 - `/register`
 - `/profile`

- Suite walutowe, domenowe, billingowe, portalowe, raportowe i service page przeszły w pełnym przebiegu.

Migracje i narzędzia jakości

- Migracje przeszły na świeżej, osobnej bazie SQLite `database/phase6-validation-*.sqlite`; plik testowy został usunięty po walidacji.
- `vendor\bin\pint --test --dirty` - OK.
- `npm run lint` - OK.
- `npm run build` - OK; Vite pokazuje tylko istniejące ostrzeżenia deprecation dla React plugin/rolldown.
- `git diff --check` - OK; tylko ostrzeżenia Git o normalizacji CRLF w istniejących plikach.

Translation check

- `bash` nie jest dostępny w lokalnej sesji PowerShell.
- Równoważnik `php artisan lang:missing --sync --force` nie może być uruchomiony, bo komenda `lang:missing` nie istnieje w tej instalacji.
- Pomocniczy skan przez `rg` działa; pokazuje głównie istniejące polskie treści/komentarze w mailach, komendach i usługach.

Skan hardcoded walut

- Brak trafień dla najważniejszych wzorców w realnych finansowych flow:
 - `money('GBP')`
 - `prefix('£')`
 - `Revenue (£)`
 - niebezpieczne agregacje `sum('total') / sum('retail_price')`
- Pozostałe świadome trafienia:
 - `resources/views/filament/actions/op-prices-diff.blade.php` - hurtowy diff OpenProvider w bazowej walucie importu.
 - `app/Console/Commands/CrmDemoSend.php` - demo/scenariusz pokazowy.
 - historyczne/testowe scenariusze demonstracyjne mogą nadal używać konkretnych kwot, ale nie są seedem publicznej oferty.

Faza 7 - wykonane

Publiczne ceny usług

- Dodano `database/migrations/2026_06_13_000003_add_price_book_to_service_items_table.php`.
- `service_items` ma teraz:
 - `price_from_prices` jako JSON price book per waluta (GBP, EUR, PLN);
 - `price_from_period` (one_time, monthly, yearly);
 - legacy `price_from` zostaje fallbackiem dla starych rekordów.
- `ServiceItemService` mapuje publiczny payload ceny według locale:
 - `pl` -> PLN;
 - `pt` -> EUR;
 - `en` -> GBP;

- fallback do domyslnego **GBP**, jesli cena lokalna nie istnieje.
- **ServiceController** i **WelcomeController** uzywaja wspolnego mappera, wiec strona glowna, **/services** i **/services/{slug}** dostaja taki sam payload ceny.
- **ServiceItemResource** w Filament ma sekcje **Structured Price Book** dla **GBP/EUR/PLN** i podglad price booka w tabeli.
- **ServiceItemSeeder** generuje ceny **GBP/EUR/PLN** z **GBP** przez **CurrencyPriceCalculator**.

Kalkulator kosztow

- Dodano **database/migrations/2026_06_13_000004_allow_calculator_pricing_multi_currency.php**.
- **calculator_pricing** dopuszcza teraz wiele rekordow dla tego samego **category + key**, rozroznionych przez **currency**.
- Dodano **CalculatorPricingPayloadService**, ktory wybiera rekordy kalkulatora dla waluty wynikajacej z locale, ale fallbackuje caly kalkulator do **GBP**, jesli lokalny price book nie jest kompletny.
- **WelcomeController** i **KalkulatorController** korzystaja ze wspolnego payload service.
- **CalculatorPricingSeeder** tworzy rekordy **GBP/EUR/PLN** z **GBP** wedlug tej samej reguly zaokraglania kursu w gore do **0.10**.
- Dodano **pages_addon.additional_page**, zeby doplata za podstrony przestala byc zaszyta jako **£80** w JS.
- **CalculatorPricingResource** i **CalculatorPricingTableWidget** obsluguja kategorie **pages_addon** i pokazuja walute rekordu.

Frontend

- Dodano **resources/js/utils/servicePrice.js** do skladania publicznej etykiety ceny z kwoty, waluty i okresu.
- **Services.jsx**, **Services/Index.jsx** i **Services/Show.jsx** formatuja ceny przez **useCurrency()**.
- **CostCalculator.jsx** i **CostCalculatorV2.jsx** formatuja wyniki i podpowiedzi przez **useCurrency()** oraz walute z payloadu pricingu.
- **resources/js/utils/currency.js** przyjmuje **minimumFractionDigits / maximumFractionDigits**, wiec marketingowe ceny moga pokazywac **£799** zamiast **£799.00**.
- Seedery tekstow kalkulatora dodaja **per_month** i **pages_addon_label**.

Testy i walidacja

- Dodano **tests/Feature/Currency/ServiceItemCurrencyTest.php**:
 - **/services** wybiera **PLN** przy **locale=pl**;
 - detal usluga wybiera **EUR** przy **locale=pt**;
 - detal usluga fallbackuje do **GBP**, gdy lokalna cena nie istnieje;
 - **/calculate** wybiera cennik **PLN** przy **locale=pl**;
 - **/calculate** fallbackuje caly pricing do **GBP**, gdy lokalny price book nie jest kompletny.
- **php artisan test tests\Feature\Currency\ServiceItemCurrencyTest.php** - OK, 5 testow / 82 asercje.
- **php artisan test tests\Unit\Currency tests\Feature\Currency tests\Unit\Billing\PlanServiceTest.php tests\Unit\Domain\DomainPricingServiceTest.php** - OK, 45 testow / 166 asercji.
- Swieza migracja i seed na tymczasowym SQLite - OK.

- `vendor\bin\pint --dirty` - OK.
 - `npm run lint` - OK.
 - `npm run build` - OK; Vite pokazuje tylko istniejące ostrzeżenia deprecation/plugin timing.
-

Faza 8 - wykonane

Seedery i publiczne copy

- `ServiceItemSeeder` nie seeduje już publicznych tekstów typu `from £..., £.../mo` ani legacy etykiet `price_from`; ceny zostają w `price_from_prices` (GBP/EUR/PLN) i `price_from_period`.
- `SiteSectionSeeder` ma strukturalny fallback price book dla kart usług na stronie głównej, a FAQ i portfolio nie zawierają już symbolu £.
- `DomainLegalPagesSeeder` i `DomainLegalPagesExtraSeeder` mówią o walucie pokazanej przy checkout/portalu klienta zamiast stalego GBP.
- `BriefingTemplateSeeder`, `LeadSeeder`, `PortfolioProjectSeeder`, `CalculatorStringsSeeder`, `DomainPriceListSeeder` i komentarze `PlanSeeder` zostały oczyszczone z mylących hardcoded przykładów funta.
- Seedowane notatki hurtowe domen zachowują informacyjny kontekst GBP, ale bez symbolu £ i bez wyglądu publicznej ceny dla klienta.

Frontend fallbacki

- `resources/js/utils/servicePrice.js` potrafi teraz złożyć etykiety ceny bezpośrednio z `price_from_prices`, jeśli payload jest fallbackiem z seeda sekcji.
- `Services.jsx`, `Services/Index.jsx` i `Services/Show.jsx` przekazują aktywną walutę do `servicePriceLabel`.
- Fallbacki `Marketing/Faq.jsx`, `Marketing/Portfolio.jsx` i stary `Marketing/CostCalculator.jsx` nie zawierają już statycznych kwot w GBP.
- Adminowe helpery/placeholdery dla publicznych usług nie sugerują już £799 jako domyślnego wzorca.

Testy i walidacja

- `rg` dla £, `charged in GBP`, `from £`, `od £`, `a partir de £` w `database/seeder`s - brak trafień.
 - `rg` dla tych samych wzorców w publicznych fallbackach marketingowych i zasobach admina usług - brak trafień.
 - `npm run lint` - OK.
 - `npm run test -- resources/js/tests/currency.test.js` - OK, 4 testy.
 - `php artisan test tests\Unit\Currency tests\Future\Currency tests\Unit\Billing\PlanServiceTest.php tests\Unit\Domain\DomainPricingServiceTest.php` - OK, 45 testów / 166 asercji.
 - `npm run build` - OK; Vite pokazuje tylko istniejące ostrzeżenia deprecation dla React/Rolldown.
 - Świeża migracja i seed na tymczasowej bazie SQLite - OK.
 - `git diff --check` - OK; tylko ostrzeżenia Git o normalizacji CRLF w istniejących plikach.
-

Raport końcowy

Multi-currency zostało wdrożone end-to-end dla **GBP**, **EUR** i **PLN**, z waluta wybierana centralnie na podstawie aktualnego **locale**.

Zakres zakończonych prac:

- centralna konfiguracja walut, resolver i formatowanie po stronie PHP oraz JS;
- Inertia shared props z aktualna waluta i metadanymi walut;
- domyślna waluta nowych leadów, klientów, projektów, ofert, faktur, kontraktów, płatności i zamówień domen;
- multi-currency dla domen, cenników TLD, zamówień, odnowień i widoków domenowych;
- multi-currency dla planów SaaS przez **plan_prices** i Stripe Price IDs per waluta;
- raporty, dashboardy, PDF-y i e-maile formatują kwoty według waluty rekordu albo grupują wartości per waluta;
- publiczne ceny usług i kalkulator kosztów korzystają ze strukturalnych price booków **GBP/EUR/PLN**;
- seedery i publiczne fallbacki marketingowe zostały oczyszczone z hardcoded copy w GBP.

Najważniejsze decyzje:

- **pl** -> **PLN**, **pt** -> **EUR**, **en** -> **GBP**;
- fallback waluty pozostaje **GBP** albo skonfigurowana waluta domyślna;
- ceny dla innych walut są generowane z GBP z kursem zaokrąglanym w górę do najbliższego **0.10**, a wynik jest zapisywany jako stabilny price book;
- istniejące/histeryczne dokumenty finansowe zachowują zapisaną walutę i nie są przeliczane.

Ostatnia walidacja:

- **npm run lint** - OK;
- **npm run test -- resources/js/tests/currency.test.js** - OK;
- **php artisan test tests\Unit\Currency tests\Feature\Currency tests\Unit\Billing\PlanServiceTest.php tests\Unit\Domain\DomainPricingServiceTest.php** - OK, 45 testów / 166 asercji;
- **npm run build** - OK;
- świeże **migrate:fresh --seed --force** na tymczasowym SQLite - OK;
- **git diff --check** - OK;
- skan seedów i publicznych fallbacków dla **£**, **charged in GBP**, **from £**, **od £**, **a partir de £** - brak trafień.

Status etapów

- ☒ Delta analysis
- ☒ Plan
- ☒ Akceptacja planu
- ☒ Implementacja Fazy 1
- ☒ Walidacja Fazy 1
- ☒ Implementacja Fazy 2
- ☒ Walidacja Fazy 2
- ☒ Implementacja Fazy 3
- ☒ Walidacja Fazy 3

- ☒ Implementacja Fazy 4
- ☒ Walidacja Fazy 4
- ☒ Implementacja Fazy 5
- ☒ Walidacja Fazy 5
- ☒ Walidacja Fazy 6
- ☒ Implementacja Fazy 7
- ☒ Walidacja Fazy 7
- ☒ Implementacja Fazy 8
- ☒ Walidacja Fazy 8
- ☒ Raport koncowy